# Color Block Height Anomaly Sorting

**Color Block Height Anomaly Sorting**

1. Course Content
2. Preparation
3. Case Demonstration
4. Color Block Color Calibration
5. Core Source Code Analysis

## 1. Course Content

- This feature implements the program acquiring images through the camera and, according to the key input, selecting the color of the color block to be sorted. The program will recognize the color blocks that meet the requirements and lower its gripper to grasp the color blocks whose height is above 4 centimeters, and finally place them in a set position.

> **Important**
>
> The grasping point calculation methods differ between the depth camera and the USB monocular camera: the depth camera obtains the spatial coordinates of the target center point through the depth map, while the monocular camera performs PnP solving to obtain the spatial position of the target center point using a known side length (0.03m)

**The wooden block used in this section:** 30x30x30mm, 30x30x60mm color blocks

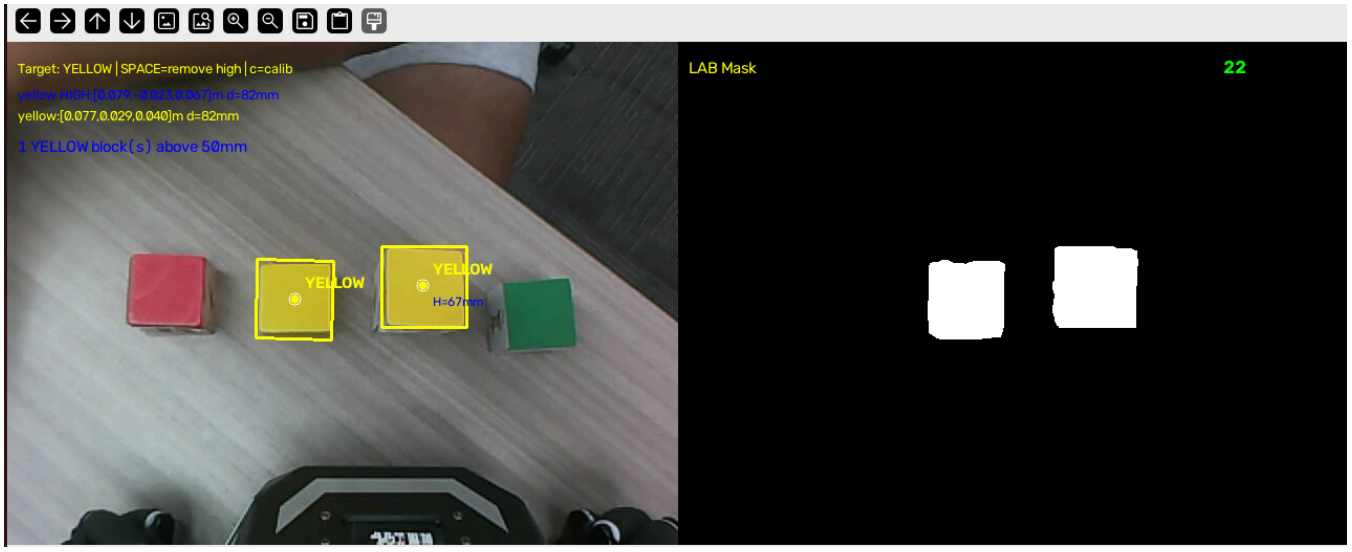
## 2. Preparation

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=color_remove_height
```

## 3. Case Demonstration

- Key R: detect and sort the red block
- Key G: detect and sort the green block
- Key B: detect and sort the blue block
- Key Y: detect and sort the yellow block

- Here, detecting the yellow block is taken as an example. The image will display all yellow blocks and the heights of target blocks whose height is above 5 centimeters




- After pressing the spacebar, the chassis will first align with the target. After the chassis alignment with the target is complete, grasping is performed. After grasping is complete, the block is placed in different positions according to the color block's color. The robotic arm then resets, and pressing the spacebar again allows the next sorting round.

# 4. Color Block Color Calibration

Refer to the color calibration steps in the previous section's tutorial 【Color Block Color Sorting】; the calibration method is the same.

# 5. Core Source Code Analysis

- Program path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/color_remove_height.py
```

**_process_remove_height function (starting the sorting thread):**

- `self._grasp_in_progress`: grasping status flag; if grasping is already in progress, returns directly to prevent repeated triggering
- `color_name = self.selected_block_color or self._target_color`: the target color preferentially uses the color selected by the user's key input, and falls back to the currently detected color when none is selected
- `threading.Thread(target=self._execute_remove_height, daemon=True)`: starts a background daemon thread to execute the removal task, avoiding blocking the main thread

**_get_fresh_block_by_center function (matching color blocks by center coordinates):**

This function finds, in the latest detection results, the color block with the specified color whose center coordinates are closest to the target, used for cross-frame matching of the same target.

- `self._latest_blocks`: traverses the list of all the latest detected color blocks
- `block['color'] != color_name`: filters out color blocks of non-target colors
- `block.get('measure_pose')`: obtains the 3D pose of the color block in the measure coordinate frame; skips if there is no pose
- `cx, cy = block['center']`: obtains the color block's center pixel coordinates
- `dsq = (cx - tx) ** 2 + (cy - ty) ** 2`: calculates the squared distance to the target center coordinates (avoiding a square root operation)
- Returns the color block with the smallest distance and its pose `(best_block, best_pose)`

**_iterative_align function (iterative alignment):**

This function adjusts the chassis position through multiple iterations to make the robot aim at the target color block, ensuring that the relative position between the robotic arm end-effector and the target meets the preset conditions during grasping.

- `tolerance = 0.01`: alignment tolerance (meters); when the position error is less than this value, alignment is considered complete
- `required_stable = 2`: alignment must be detected as stable for 2 consecutive times before returning, preventing a single accidental satisfaction of the condition
- `max_iterations = 12`: maximum number of iterations; after exceeding this, alignment is abandoned and the current pose is returned
- `max_step = 0.04`: maximum distance of a single move (meters), limiting the adjustment amplitude each time to avoid overshoot or target loss
- `_get_fresh_block_by_center(color_name, block_center)`: re-detects the color block at each iteration, and ensures tracking of the same target through color + center coordinate matching
- `delta_x = pose[0] - grasp_distance_x`: calculates the deviation between the current x direction and the expected grasping distance
- `y_needs = abs(pose[1]) > grasp_distance_y + tolerance`: determines whether the y-direction deviation exceeds the tolerance
- `stable_count`: stability counter; increments when the alignment condition is satisfied consecutively, otherwise resets
- `move_by_odom(distance=..., direction=...)`: calls the chassis odometry Action to move the specified distance; the direction is determined by the sign of the deviation (forward/left/right)
- After each move, wait 2 seconds for the chassis to stabilize, then re-detect the pose

```
    def _process_remove_height(self, blocks):
        """Start the sorting thread: remove all selected-color blocks whose height exceeds
remove_height"""
        if self._grasp_in_progress:
            return

        # The target color selection persists; prefer the selected color, fall back to the
current detection color
        color_name = self.selected_block_color or self._target_color
        if color_name is None:
```

```
            self.get_logger().warn("SPACE ignored: no target color selected")
            return

        self._grasp_in_progress = True
        threading.Thread(
            target=self._execute_remove_height,
            args=(color_name,),
            daemon=True
        ).start()

    def _get_fresh_block_by_center(self, color_name, target_center):
        """Find a specific block in _latest_blocks by color + nearest center coordinates

        Args:
            color_name: target color
            target_center: (cx, cy) expected center coordinates

        Returns:
            (block_dict, [mx, my, mz]) or (None, None)
        """
        best_block, best_pose, best_dist = None, None, float('inf')
        tx, ty = target_center
        for block in self._latest_blocks:
            if block['color'] != color_name:
                continue
            pose = block.get('measure_pose')
            if pose is None:
                continue
            cx, cy = block['center']
            dsq = (cx - tx) ** 2 + (cy - ty) ** 2
            if dsq < best_dist:
                best_dist = dsq
                best_block = block
                best_pose = pose
        return best_block, best_pose

    def _iterative_align(self, color_name, block_center):
        """Iterative chassis alignment: re-detect the block pose after each small step until
the grasp distance is met and stable

        Args:
            color_name: target color
            block_center: (cx, cy) center pixel coordinates of the target block, used for
cross-frame matching
        """
        tolerance = 0.01
        required_stable = 2
        max_iterations = 12
        max_step = 0.04  # at most 4cm per step to keep the block in view
        stable_count = 0

        for iteration in range(max_iterations):
            if not rclpy.ok():
```

```
                return None

            time.sleep(0.5)
            block, pose = self._get_fresh_block_by_center(color_name, block_center)
            if block is None or pose is None:
                self.get_logger().warn(
                    f"[Iter {iteration+1}] {color_name} block not visible"
                )
                stable_count = 0
                continue

            delta_x = pose[0] - self.grasp_distance_x
            x_needs = delta_x > tolerance
            y_needs = abs(pose[1]) > self.grasp_distance_y + tolerance

            if not x_needs and not y_needs:
                stable_count += 1
                if stable_count >= required_stable:
                    self.get_logger().info(
                        "Chassis alignment stable, returning latest pose for IK"
                    )
                    return pose
                time.sleep(0.3)
                continue
            stable_count = 0

            if x_needs:
                move_x = min(delta_x, max_step)
                self.move_by_odom(distance=move_x, direction='forward')

            if y_needs:
                move_y = min(
                    abs(pose[1]) - self.grasp_distance_y, max_step
                )
                direction = 'left' if pose[1] > 0 else 'right'
                self.move_by_odom(distance=move_y, direction=direction)
            time.sleep(2.0)

        self.get_logger().warn(
            f"Alignment did not converge after {max_iterations} iterations"
        )
        _, pose = self._get_fresh_block_by_center(color_name, block_center)
        return pose
```

- Parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

- When there is a grasping deviation, the grasp_offset can be adjusted

```yaml
color_remove_height:
  ros__parameters:
    color_grasp_distance: [0.078, 0.0]     # x,y axis alignment distance (meters)
    grasp_offset: [0.0,0.0,0.0,85.0]     # x,y,z axis offset distance and included angle, compensating for servo backlash, link rigidity degradation, and special grasping requirements
    gripping_angle: 140                      # gripper grasping angle, in degrees, integer
    observation_pose: [91, 95, 35, -34, 90, 30]
    square_size: 0.03
    usb_camera_measure: [0.012, 0.0, 0.013]
```